

Project Title: Research and Implementation of an Enhanced Liquid Pouring Robot System Based on VLA

1. Abstract

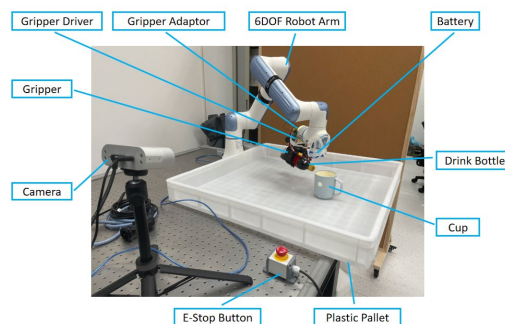
- (1) Background: Challenges of Embodied AI in fine-grained manipulation tasks.
- (2) Core Solution: Implementing liquid pouring using the OpenVLA-7B model with LoRA fine-tuning.
- (3) Key Achievements: Validating VLA action prediction and execution in specific scenarios through real-world data cleaning and training.

2. Introduction

- (1) Task Context: action command's understanding and spatial perception requirements for pouring tasks.
- (2) Limitations: Inflexibility of traditional pre-programmed trajectories and significant domain gaps in simulation-only data.
- (3) Contributions: Proposing a high-quality, real-world teleoperation data-driven VLA fine-tuning scheme.

3. System Architecture & Hardware Setup

- (1) Algorithmic Backbone: OpenVLA-7B large model architecture.
- (2) Robot Platform:



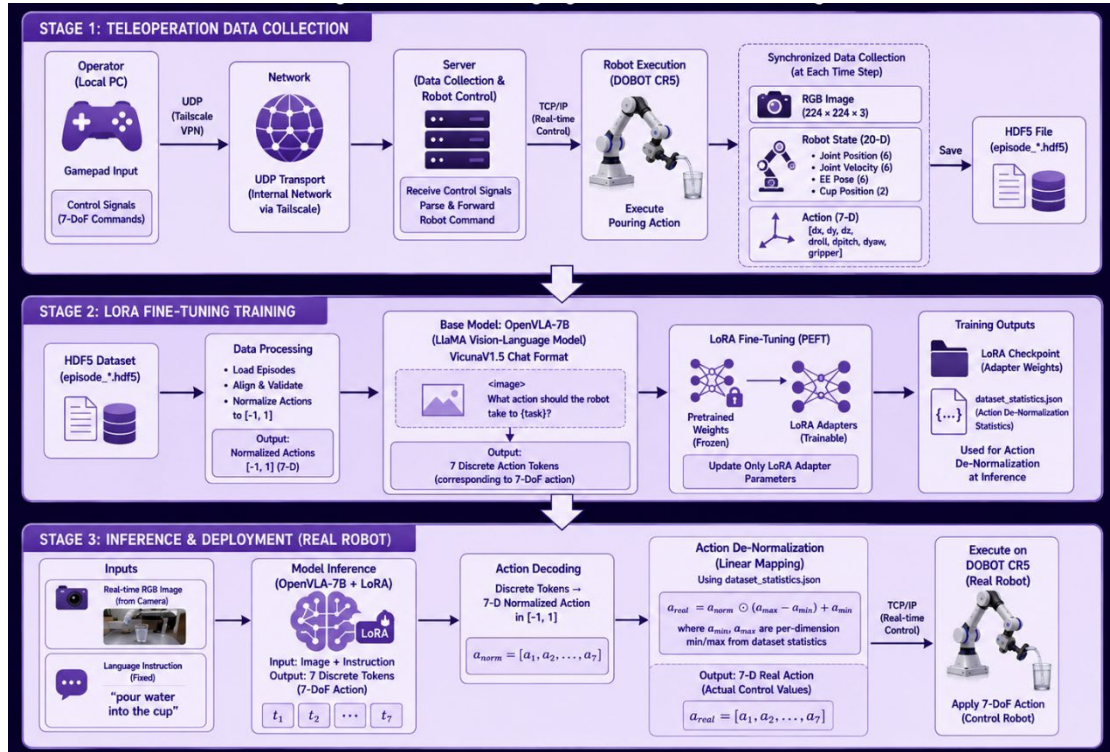
Picture 1. Physical Robot Platform Setup

Picture 1 shows the physical robot platform setup of our project. The key components consist of the following items:

- 1) DOBOT Nova5: a 6 DOF robot arm, to provide the desired pose and position
- 2) the gripper: has two status, namely open and close
- 3) the cup: a normal cup, better to be non-transparent and ship-resistant
- 4) the drink bottle: here, we use a Coca Cola drink bottle, for its color is dark and easy to be distinguished from the environment
- 5) the camera

We designed and 3D printed a gripper adaptor to mount the gripper onto the robot arm. The gripper's driver and battery are bonded on the adaptor. To prevent drink's leakage, we use a large plastic pallet to hold the cup and drink bottle, which has been proven to be a wise choice during data collecting and testing. Besides, a E-stop button is placed aside, preparing for the emergency situations.

(3) Computing Platform: Server-side training and deployment plans for edge devices (Qualcomm RB8) and RTX3090.



4. Data Engineering (Key Focus)

(1) Real-world Data Collection: Utilizing gamepad-based teleoperation to capture actual task trajectories:

In teleoperation, the robot and the gripper's motions can be simplified and extracted into these ones in the left column of the following table. In order to boost the data

collection efficiency, we built the matching relationship between the robot’s motion(facing the robot from the table side) and the gamepad’s button as Table 1.

The System’s Motion	Gamepad Button
Left in Human Eyes’ Coordinate	Left in Left Panel
Right in Human Eyes’ Coordinate	Right in Left Panel
Forward in Human Eyes’ Coordinate	Up in Left Panel
Backward in Human Eyes’ Coordinate	Down in Left Panel
Clockwise Rotation in Nova5’s Last Joint	Left in Right Panel
Anticlockwise Rotation in Nova5’s Last Joint	Right in Right Panel
Rising in Human Eyes’ Coordinate	Up in Right Panel
Descending Left in Human Eyes’ Coordinate	Down in Right Panel
Pitch in End-effector’s Coordinate	Left Front Button
Anti-Pitch in End-effector’s Coordinate	Right Front Button
Yaw in End-effector’s Coordinate	Left Bottom Button
Anti-yaw in End-effector’s Coordinate	Right Bottom Button

Table 1. Matching relationship between motions and buttons

Totally, 85 sets of task trajectories were collected by the teleoperation method, in which the positions of drink bottle and cup are fixed points.

(2) Data Cleaning & Pre-processing:

The teleoperation data in this project was recorded at 5 Hz, with each episode stored as an HDF5 file containing 224×224 RGB images, 7-DoF end-effector delta actions ([dx, dy, dz, droll, dpitch, dyaw, gripper]), and the full robot state vector. Before being used for training, the raw data goes through the following cleaning and alignment steps.

① Filtering static frames to improve training efficiency:

During teleoperation, operators naturally introduce many "static frames" — timesteps where the robot arm is barely moving, such as brief pauses before grasping or hesitations at phase transitions. These frames carry no useful action signal for training. Instead, they repeatedly expose the model to examples of (observation → near-zero action), which can push the model toward always predicting small actions regardless of the visual input — a key contributing factor to action collapse.

To address this, we define a static frame as one that simultaneously satisfies both of the following conditions, and remove it from the episode:

- The L2 norm of the position delta is below 1.5 mm ($\|[dx, dy, dz]\|_2 < 0.0015$ m)
- The L2 norm of the rotation delta is below 0.008 rad

The only exception is the frame at which the gripper changes state (a sign flip in the gripper channel). This frame is kept even if the arm displacement is near zero, since it captures the exact moment of grasping or releasing — a semantically critical event that should not be discarded.

After filtering, the proportion of training-useful frames in each episode increases, and the risk of the model overfitting to meaningless near-zero actions is reduced.

② Aligning action data specific to VLA model requirements:

The raw HDF5 data format is not directly compatible with what OpenVLA expects, and a few alignment steps are needed.

The first is the gripper channel remapping. During collection, the gripper state is encoded as +1 (open) and −1 (closed), whereas OpenVLA follows the Open-X Embodiment convention of 1.0 (open) and 0.0 (closed). We convert using $(val + 1) / 2$ and then discretize to $\{0.0, 1.0\}$ with a threshold of 0.5. The discretization step matters here: the robot's gripper controller interpolates smoothly over roughly 0.3 seconds during actuation, producing intermediate values that are neither fully open nor fully closed. Without discretization, these soft-transition frames would confuse the model about the gripper's intent.

Otherwise, to conform to the RLDS data format, each step is supplemented with the required fields: `is_first`, `is_last`, `is_terminal`, `reward` (1.0 on the final step, 0.0 otherwise), `discount` (1.0 throughout), `language_instruction`, and `language_embedding` (a 512-dimensional zero vector — OpenVLA re-encodes the text internally during training, so zeros suffice here).

5. Model Training & Fine-tuning

(1) Fine-tuning Strategy: Employing LoRA (Low-Rank Adaptation) to reduce VRAM usage and accelerate convergence:

OpenVLA-7B has approximately 7 billion parameters, making full fine-tuning prohibitively expensive in terms of GPU memory, and also prone to overfitting given the limited scale of our demonstration dataset. We therefore adopt LoRA (Low-Rank Adaptation) for parameter-efficient fine-tuning. The idea behind LoRA is to insert a pair of low-rank matrices: $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ alongside the original weight matrices. Only these adapter weights are trained while the base model remains frozen, which drastically reduces the number of trainable parameters and keeps GPU memory usage manageable.

LoRA is applied to all linear layers in the model, covering not just the attention and FFN projections in the LLaMA language backbone, but also the linear layers in the vision backbone and vision-language projector. This is broader than adapting only the language model layers, allowing the full model to better adapt to the task-specific visual-language-action distribution. The rank is set to $r=32$, $\alpha=16$ (computed as $\min(\text{rank}, 16)$), with Gaussian weight initialization.

The training hyperparameters are as follows:

Image augmentation is enabled to improve robustness against camera viewpoint and lighting variations, reducing overfitting to the specific visual conditions in the demonstrations. Training uses the AdamW optimizer with a constant learning rate, and all metrics are logged via Weights & Biases.

Regarding checkpoint saving, we modified the script so that each checkpoint saves only the LoRA adapter weights independently, without merging them into the base model. This keeps each checkpoint compact and saves faster, while also making it easier to evaluate or compare adapters from different training stages. At inference time, the selected adapter is simply loaded on top of the frozen base model.

(2) Training Objective: Training Objective: Cross-entropy loss over discretized action tokens:

Unlike typical regression-based robot learning approaches, OpenVLA does not use MSE loss. Instead, the training objective is a cross-entropy classification loss over discretized action tokens.

Each of the 7 continuous action dimensions is mapped to a discrete bin based on the dataset's action statistics (percentile-based normalization), with each bin corresponding to a specific token ID in the model's vocabulary. Predicting a full 7-DoF action is therefore equivalent to predicting 7 discrete tokens sequentially — the model is performing 7 independent classification tasks at each timestep.

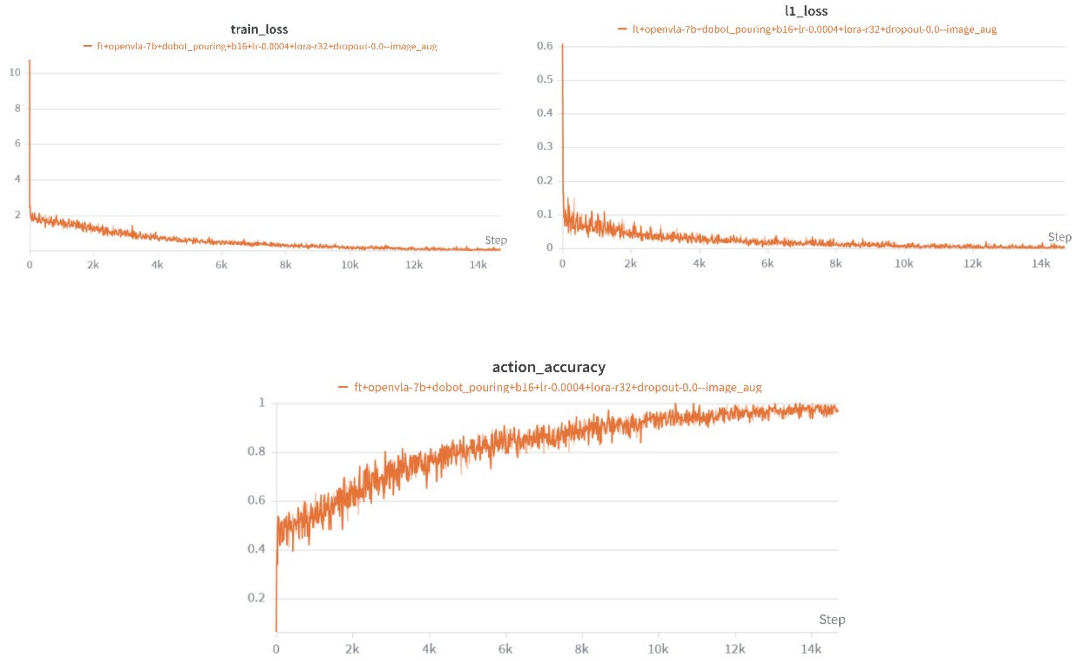
The loss is computed only on the positions of these 7 action tokens. All preceding tokens — visual patch embeddings and the language instruction prefix — are masked and excluded from gradient computation. This ensures the gradient signal comes entirely from action prediction, without interference from language generation objectives. In addition, the training script also monitors an L1 loss alongside the cross-entropy loss: the predicted action tokens are decoded back into continuous action values, and the L1 distance between predicted and ground-truth continuous actions is computed. This metric provides a more intuitive measure of actual action error than the token-level cross-entropy, making it easier to judge whether training is converging in the right direction.

6. Experimental Results & Analysis

In this project, to achieve better grasping and pouring performance with the robotic arm, multiple training experiments were conducted to progressively resolve existing issues. These primarily include the following three experiments:

(1) Experiment 1: Successful implementation of grasping and pouring cola at a fixed location.

Training Part:



In Experiment 1, we collected 88 trajectories via gamepad teleoperation for grasping and pouring a drink at a fixed position, forming the dataset used for LoRA training. After data filtering, the dataset was converted into the RLDS format. Training was conducted on an RTX 5090 (32GB) GPU on the AutoDL platform using the following command:

```
(openVLA) zjf@zjf-Precision-3680:~/pouring_VLA$ python finetune.py \  
--vla_path /root/autodl-tmp/checkpoints/openvla-7b \  
--data_root_dir /root/autodl-tmp/vla_dataset_overfit_RLDS \  
--dataset_name dobot_pouring \  
--run_root_dir /root/zjf/openvla/runs3 \  
--adapter_tmp_dir /root/zjf/openvla/adapter-tmp3 \  
--lora_rank 32 \  
--batch_size 4 \  
--grad_accumulation_steps 4 \  
--learning_rate 4e-4 \  
--image_aug True \  
--wandb_project openvla-pouring-v1 \  
--wandb_entity 2274641775-northwestern-polytechnic-university \  
--save_steps 1000 \  
--max_steps 16000
```

As illustrated in the figures, the model demonstrated a stable and favorable convergence trend throughout the LoRA fine-tuning process.

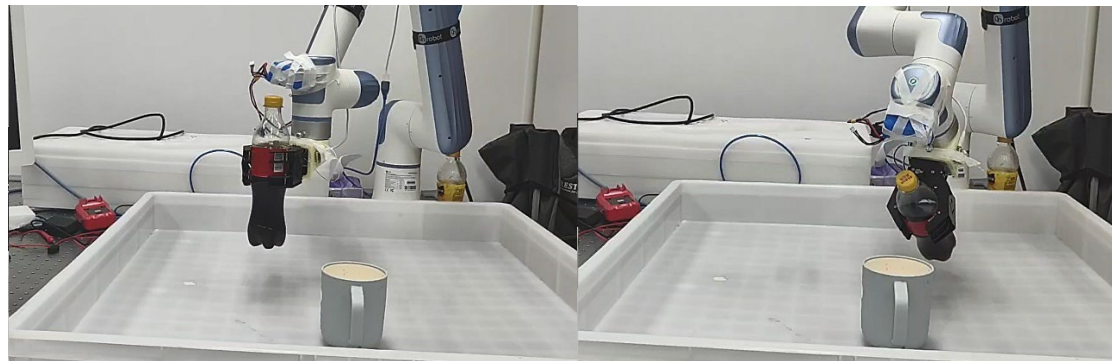
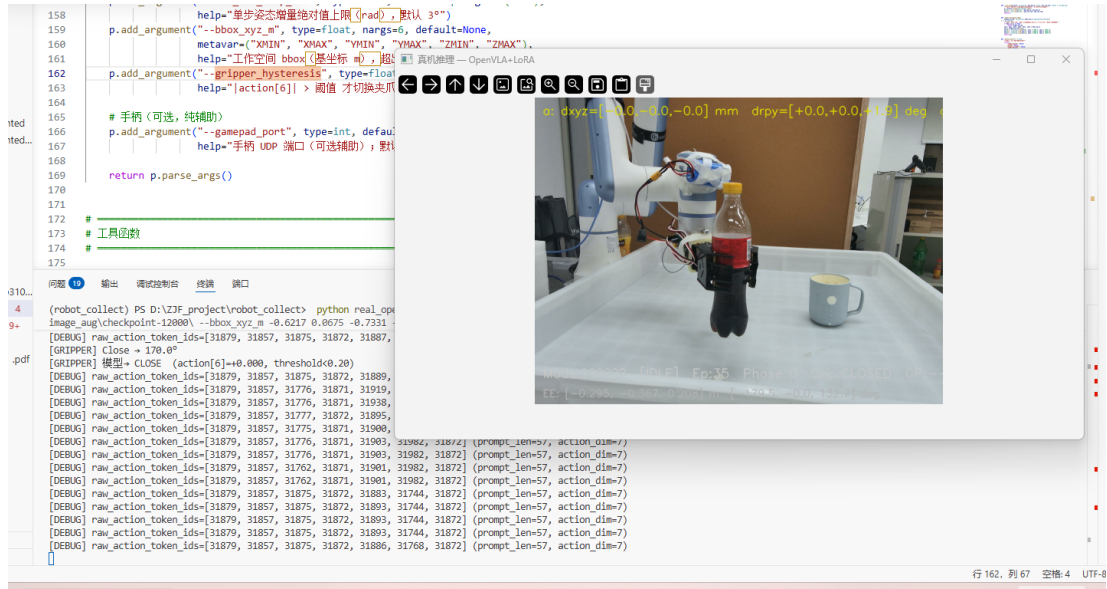
Both `train_loss` and `l1_loss` exhibited a rapid and significant decrease during the early stages of training (approximately the first 2,000 steps), indicating that the model quickly adapted to the data distribution of the pouring task. Between 8,000 and 10,000 steps, the downward trend of both loss metrics gradually flattened out and approached convergence. Ultimately, the `l1_loss` dropped to a near-zero level, demonstrating a highly precise alignment between the model's predicted continuous action vectors and the ground-truth teleoperation trajectories.

Corresponding to the loss metrics, the `action_accuracy` curve showed a steady upward trend. Around 10,000 steps, the action prediction accuracy surpassed 0.9 (90%) and stably converged to nearly 1.0 before reaching the maximum set steps (16,000 steps). These results verify that under a limited dataset (88 trajectories), the LoRA fine-tuning strategy with a rank of $r=32$ enables the OpenVLA-7B model to efficiently learn the grasping and pouring logic at fixed positions without significant underfitting.

Inference Part:

Using this trained model, we performed inference and robotic arm control on a computer equipped with an RTX 3090 GPU: By combining the LoRA checkpoint at 14,000 steps with the openVLA-7B base model, we conducted 7x1 dimensional action inference based on the current RGB image input.

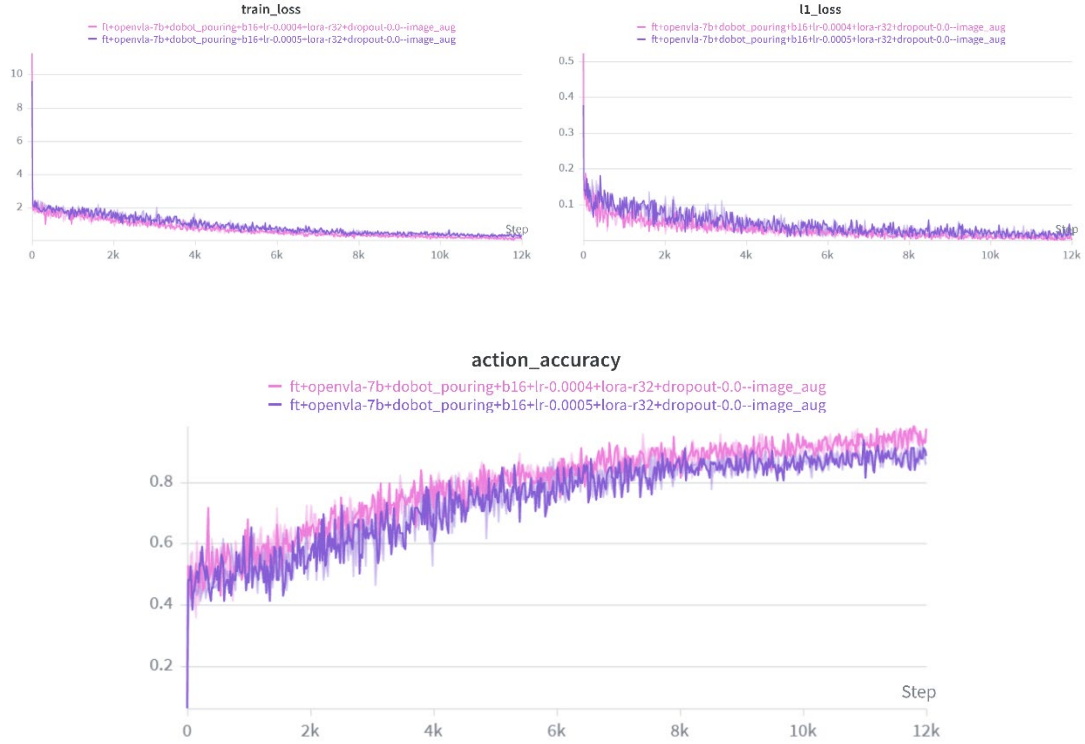
The resulting pose increments were then transmitted to the robotic arm end-effector via the `servoP` command to execute the grasping and pouring operations. However, the overall inference frequency could only be maintained between 1.2 Hz and 2 Hz, so the robotic arm's movements appeared noticeably jerky.



(2) Experiment 2&3: Grasping and pouring of cola across a specific range of positional variations, yet failing to achieve accurate grasping.

In Experiments 2 and 3, we explored grasping and pouring tasks under conditions where the beverage position varies. Similarly, approximately 140 trajectory data points were collected via teleoperation, and the RLDS dataset was generated after data filtering and preprocessing. However, due to the complexity of the task, the average trajectory length in the dataset is approximately 350 frames, which makes OpenVLA highly susceptible to long-horizon action collapse.

To mitigate this issue, we partitioned the entire trajectory into three stages, each utilizing distinct language prompts: "pick up cola," "move to cup," and "pour cola." This strategy, to a certain extent, prevents the issue of long-horizon action collapse. This constitutes the primary difference between Experiments 2 and 3: in Experiment 2, a unified language prompt, "pour cola into cup," was applied to the entire trajectory, while in Experiment 3, stage-specific prompts were used during training. As shown in the training metrics graph below, the purple curve represents Experiment 2, and the pink curve represents Experiment 3.



As can be seen from the training metrics graph above, both types of loss in the three-stage experiment decrease faster and more smoothly, and the action prediction accuracy is also higher. This is because the model only needs to learn a simple, well-defined action mapping within a shorter sequence of frames, which greatly reduces the learning difficulty.

However, although the action accuracy can reach around 90% and the loss eventually converges near 0—metrics that seemingly indicate a highly successful training process—two problems frequently occur during inference when using the LoRA models from various checkpoints of these two experiments: an inability to accurately localize the grasping position, and a tendency to exhibit repetitive, oscillatory movements. Ultimately, these issues lead to a complete failure of the grasp-and-pour process. Through repeated inference attempts and literature review, the root cause was analyzed as follows: insufficient data. For a task requiring spatial generalization, 140 trajectory data points are entirely inadequate. Even with repeated training iterations, the model merely overfits the dataset, failing completely to learn the actual mechanics of the grasp-and-pour process.

7. Discussion & Future Work

(1) Generalization Challenges: Limitations of Data Volume

Our experiments revealed that while the OpenVLA-7B model can successfully overfit to a fixed-point pouring task, achieving spatial generalization remains a significant hurdle. The current dataset of 140 real-world trajectories is insufficient for the model to learn a robust 3D representation of the workspace. Despite high action accuracy during training, the model exhibited "localization drift" and oscillatory behaviors during inference when objects were moved. This indicates that the model merely memorized specific coordinate mappings rather than understanding the underlying physics of grasping and pouring. Future improvements require scaling the dataset to at least 500 – 1,000 diverse episodes to support true spatial generalization.

(2) Action Refinement & Future Directions: The current system faces challenges regarding motion smoothness and execution speed.

Stability & Spill Prevention: Future work will focus on integrating filtering algorithms (e.g., Savitzky-Golay filters) to eliminate the inherent "jitter" observed during VLA inference, ensuring a stable pouring process without liquid spillage.

Closed-Loop Volume Control: We plan to implement a visual feedback loop that monitors the liquid level in the cup. This would allow the robot to transition from a fixed-time pouring execution to an autonomous "volume-aware" stop, enhancing the system's cognitive intelligence.

Edge Acceleration: To resolve the current 0.03 Hz bottleneck on CPU-only inference, we will continue exploring model quantization and NPU-specific optimization to enable real-time deployment on the Qualcomm AIRbox Q900.

8. Conclusion

(1) Feasibility of VLA in Liquid Pouring

This project successfully validated the feasibility of utilizing Vision-Language-Action (VLA) models for fine-grained manipulation tasks. By fine-tuning OpenVLA-7B with LoRA, we demonstrated that large-scale pre-trained models can be adapted to specific robotic skills like liquid pouring with relatively small real-world datasets. The use of stage-specific prompts effectively mitigated the "long-horizon action collapse" common in complex multi-step tasks.

(2) Necessity of High-Quality Real-World Data

A key takeaway from this project is the critical importance of data quality over quantity. We discovered that the significant domain gap between simulation (Isaac Sim) and reality made sim-to-real transfer impractical for this specific task. Consequently, transitioning to a pure real-world data strategy—coupled with rigorous

data cleaning such as "static frame filtering"—was essential for achieving a successful proof-of-concept. High-quality, teleoperated real-world demonstrations remain the most reliable foundation for large-scale Embodied AI models.

9. Team Division of Labor:

Zou Junfeng (225015057)(40%): Served as the technical lead, responsible for the overall system architecture design and inference analysis. He designed the LoRA fine-tuning pipeline, conducted the analysis of model convergence, and led the investigation into the edge deployment failures on the Qualcomm platform. Additionally, he contributed significantly to both simulated and real-world data collection.

Cao Fengyu (225010398)(30%): Primarily Responsible for the physical hardware setup and infrastructure. He designed and 3D-printed the custom gripper adaptors, integrated the safety E-stop system, and ensured the stability of the robot workspace. He also assisted in the large-scale data collection process.

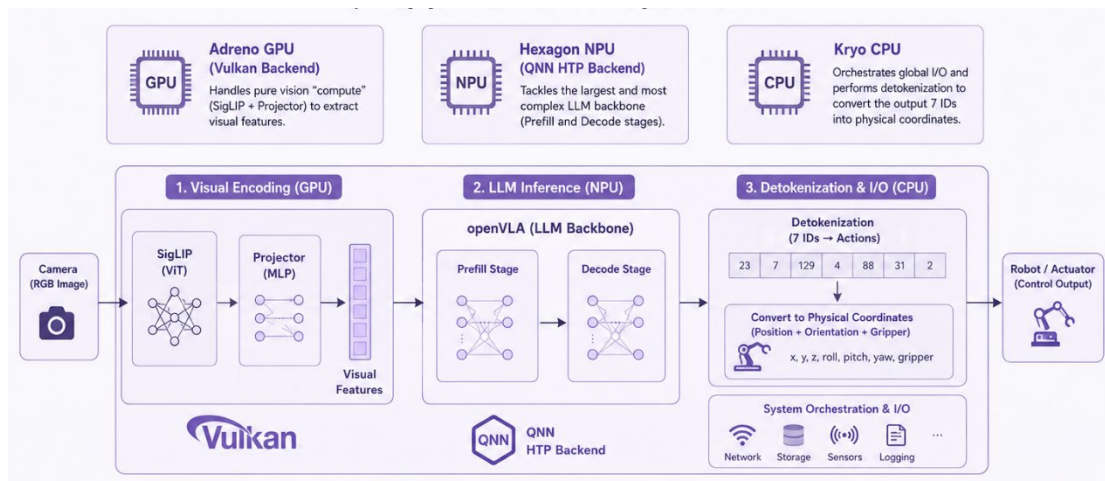
Ge Yuxuan (225040528)(30%): Primarily Responsible for data collection and comprehensive data engineering. He managed the teleoperation sessions and developed the data cleaning scripts, including the L2-norm-based static frame filtering and RLDS dataset conversion, to ensure training efficiency.

Appendix: Edge Deployment Challenges on Fogwise® AIRbox

Q900

1. Overview of the Deployment Objective

The original design of this project aimed to deploy the entire OpenVLA-7B model onto a resource-constrained edge computing device, specifically the Fogwise® AIRbox Q900 (equipped with the Qualcomm AI Engine). The goal was to achieve fully independent edge-side inference for real-time robotic control. However, significant engineering bottlenecks were encountered during the model conversion and compilation phases.



2. The Root Cause: Dynamic vs. Static Computation

The primary obstacle preventing a holistic deployment of the model is the fundamental incompatibility between the model's architecture and the Qualcomm toolchain. Qualcomm's QAIRT (Qualcomm AI Engine Direct) SDK strictly relies on static computation graphs for optimization and deployment. In contrast, the OpenVLA model, being an auto-regressive language model at its core, generates action tokens sequentially. This token-by-token generation represents a highly dynamic control flow with variable sequence lengths. The QAIRT converter is inherently incapable of compiling such dynamic computation graphs, resulting in immediate crashes when attempting to convert the entire model as a single entity.

3. Decoupled Deployment Attempts

To bypass the holistic conversion failure, we attempted to decouple the OpenVLA model and distribute its modules across the heterogeneous computing units (CPU, GPU, and NPU) of the AIRbox Q900.

Scheme 1: Cross-Hardware Heterogeneous Computing (GPU Vision + NPU LLM + CPU Control)

In this ideal decoupled architecture, the Adreno GPU would handle the pure vision computations (SigLIP + Projector) to extract visual features, while the Hexagon NPU would process the computationally massive LLM backbone. The Kryo CPU would be reserved for global I/O coordination and mathematical detokenization. Because the OpenVLA LLM is based on the LLaMA architecture, we attempted to compile llama-cpp-python using the QAIRT SDK to run on the NPU. Unfortunately, these compilation attempts consistently failed due to deep-level toolchain incompatibilities, forcing us to abandon this scheme.

Scheme 2: The Compromise Architecture (NPU Vision + CPU LLM)

As a fallback, we attempted to offload the vision feature extraction to the NPU while leaving the heavy LLM inference to the CPU. This required converting the decoupled vision module using the qairt-converter. However, during the compilation phase, the converter encountered fatal errors when processing ReshapeOp nodes (specifically, Unable to calculate ReshapeOp output shape for op node_view). The shape parameters resulted in mismatched cumulative totals for inputs and outputs, leading to a complete compilation crash.

4. Final Fallback Result: Pure CPU Inference

Having exhausted the heterogeneous computing options, we were compelled to test the entire inference pipeline purely on the Kryo CPU. While the model functionally executed without crashing, the performance was drastically compromised. The execution logs indicated an average latency of approximately 43,317 ms per step, yielding an overall inference frequency of merely 0.02 Hz.

For a delicate and dynamic physical task like liquid pouring, where real-time sensory feedback and high-frequency control loops are paramount to prevent spillage, a 43-second delay between actions is entirely unacceptable. This practical failure confirms that directly deploying 7B-parameter VLA models on current edge devices remains highly unrealistic without significant breakthroughs in dynamic graph compilation and extreme quantization techniques.